

Kube-OVN总结

总体架构

本文档将介绍 Kube-OVN 的总体架构，和各个组件的功能以及其之间的交互。

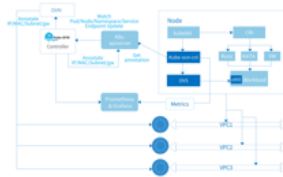
总体来看，Kube-OVN 作为 Kubernetes 和 OVN 之间的一个桥梁，将成熟的 SDN 和云原生相结合。这意味着 Kube-OVN 不仅通过 OVN 实现了 Kubernetes 下的网络规范，例如 CNI，Service 和 Networkpolicy，还将大量的 SDN 领域能力带入云原生，例如逻辑交换机，逻辑路由器，VPC，网关，QoS，ACL 和流量镜像。

同时 Kube-OVN 还保持了良好的开放性可以和诸多技术方案集成，例如 Cilium，Submariner，Prometheus，KubeVirt 等等。



Kube-OVN 的组件可以大致分为三类：

- 上游 OVN/OVS 组件。
- 核心控制器和 Agent。
- 监控，运维工具和扩展组件。



上游 OVN/OVS 组件

该类型组件来自 OVN/OVS 社区，并针对 Kube-OVN 的使用场景做了特定修改。OVN/OVS 本身是一套成熟的管理虚机和容器的 SDN 系统，我们强烈建议对 Kube-OVN 实现感兴趣的用户先去读一下 [ovn-architecture\(7\)](#) 来了解什么是 OVN 以及如何和它进行集成。Kube-OVN 使用 OVN 的北向接口创建和调整虚拟网络，并将其中的网络概念映射到 Kubernetes 之内。

所有 OVN/OVS 相关组件都已打包成对应镜像，并可在 Kubernetes 中运行。

ovn-central

ovn-central Deployment 运行 OVN 的管理平面组件，包括 **ovn-nb**，**ovn-sb**，和 **ovn-northd**。

- **ovn-nb**：保存虚拟网络配置，并提供 API 进行虚拟网络管理。**kube-ovn-controller** 将会主要和 **ovn-nb** 交互配置虚拟网络。
- **ovn-sb**：保存从 **ovn-nb** 的逻辑网络生成的逻辑流表，以及各个节点的实际物理网络状态。
- **ovn-northd**：将 **ovn-nb** 的虚拟网络翻译成 **ovn-sb** 中的逻辑流表。

多个 **ovn-central** 实例会通过 Raft 协议同步数据保证高可用。

ovs-ovn

ovs-ovn 以 DaemonSet 形式运行在每个节点，在 Pod 内运行了 **openvswitch**，**ovsdb**，和 **ovn-controller**。这些组件作为 **ovn-central** 的 Agent 将逻辑流表翻译成真实的网络配置。

核心控制器和 Agent

该部分为 Kube-OVN 的核心组件，作为 OVN 和 Kubernetes 之间的一个桥梁，将两个系统打通并将网络概念进行相互转换。大部分的核心功能都在该部分组件中实现。

kube-ovn-controller

该组件为一个 Deployment 执行所有 Kubernetes 内资源到 OVN 资源的翻译工作，其作用相当于整个 Kube-OVN 系统的控制平面。 `kube-ovn-controller` 监听了所有和网络功能相关资源的事件，并根据资源变化情况更新 OVN 内的逻辑网络。主要监听的资源包括：Pod，Service，Endpoint，Node，NetworkPolicy，VPC，Subnet，Vlan，ProviderNetwork。

以 Pod 事件为例， `kube-ovn-controller` 监听到 Pod 创建事件后，通过内置的内存 IPAM 功能分配地址，并调用 `ovn-central` 创建逻辑端口，静态路由和可能的 ACL 规则。接下来 `kube-ovn-controller` 将分配到的地址，和子网信息例如 CIDR，网关，路由等信息写会到 Pod 的 annotation 中。该 annotation 后续会被 `kube-ovn-cni` 读取用来配置本地网络。

kube-ovn-cni

该组件为一个 DaemonSet 运行在每个节点上，实现 CNI 接口，并操作本地的 OVS 配置单机网络。

该 DaemonSet 会复制 `kube-ovn` 二进制文件到每台机器，作为 `kubelet` 和 `kube-ovn-cni` 之间的交互工具，将相应 CNI 请求发送给 `kube-ovn-cni` 执行。该二进制文件默认会被复制到 `/opt/cni/bin` 目录下。

`kube-ovn-cni` 会配置具体的网络来执行相应流量操作，主要工作包括：

1. 配置 `ovn-controller` 和 `vswitchd`。
2. 处理 CNI add/del 请求：
 - a. 创建删除 veth 并和 OVS 端口绑定。
 - b. 配置 OVS 端口信息。
 - c. 更新宿主机的 iptables/ipset/route 等规则。
3. 动态更新容器 QoS。
4. 创建并配置 `ovn0` 网卡联通容器网络和主机网络。
5. 配置主机网卡来实现 Vlan/Underlay/EIP 等功能。
6. 动态配置集群互联网关。

监控，运维工具和扩展组件

该部分组件主要提供监控，诊断，运维操作以及和外部对接，对 Kube-OVN 的核心网络能力进行扩展，并简化日常运维操作。

kube-ovn-speaker

该组件为一个 DaemonSet 运行在特定标签的节点上，对外发布容器网络的路由，使得外部可以直接通过 Pod IP 访问容器。

更多相关使用方式请参考 [BGP 支持](#)。

kube-ovn-pinger

该组件为一个 DaemonSet 运行在每个节点上收集 OVS 运行信息，节点网络质量，网络延迟等信息，收集的监控指标可参考 [Kube-OVN 监控指标](#)。

kube-ovn-monitor

该组件为一个 Deployment 收集 OVN 的运行信息，收集的监控指标可参考 [Kube-OVN 监控指标](#)。

kubectl-ko

该组件为 kubectl 插件，可以快速运行常见运维操作，更多使用请参考 [kubectl 插件使用](#)。

Underlay 流量拓扑

同节点同子网

内部逻辑交换机直接交换数据包，不进入外部网络。



跨节点同子网

数据包经由节点网卡进入外部交换机，由外部交换机进行交换。



同节点不同子网

数据包经由节点网卡进入外部网络，由外部交换机及路由器进行交换和路由转发。



此处 br-provider-1 和 br-provider-2 可以是同一个 OVS 网桥，即多个不同子网可以使用同一个 Provider Network。

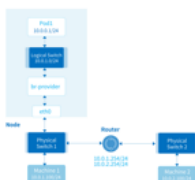
跨节点不同子网

数据包经由节点网卡进入外部网络，由外部交换机及路由器进行交换和路由转发。



访问外部

数据包经由节点网卡进入外部网络，由外部交换机及路由器进行交换和路由转发。



节点与 Pod 之间的通信大体上也遵循此逻辑。

无 Vlan Tag 下总览



多 VLAN 总览



Pod 访问 Service IP

Kube-OVN 为每个 Kubernetes Service 在每个子网的逻辑交换机上配置了负载均衡。当 Pod 通过访问 Service IP 访问其它 Pod 时，会构造一个目的地址为 Service IP、目的 MAC 地址为网关 MAC 地址的网络包。网络包进入逻辑交换机后，负载均衡会对网络包进行拦截和 DNAT 处理，将目的 IP 和端

口修改为 Service 对应的某个 Endpoint 的 IP 和端口。由于逻辑交换机并未修改网络包的二层目的 MAC 地址，网络包在进入外部交换机后仍然会送到外部网关，此时需要外部网关对网络包进行转发。

Service 后端为同节点同子网 Pod



Service 后端为同节点不同子网 Pod

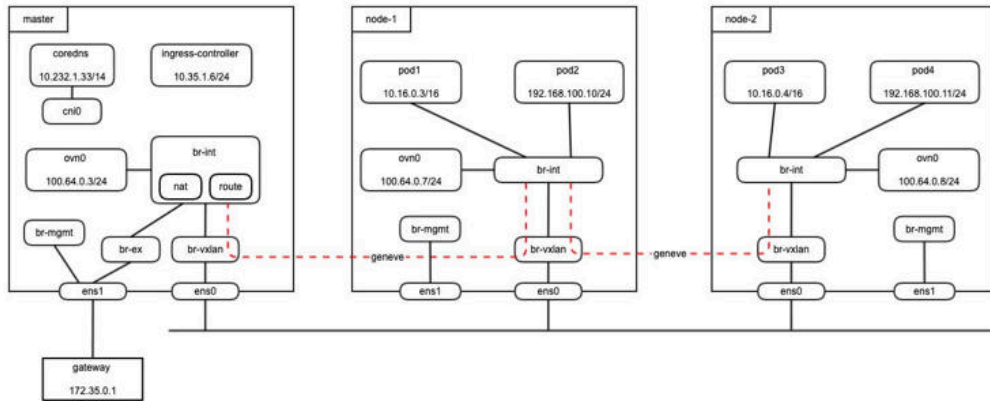


参考文档

[Kube-OVN 文档](#)

安全容器产品使用情况

- 1、产品中LB svc是基于独享型负载均衡服务实现的，与kube-ovn的LB svc实现不同。
- 2、社区文档中提到：VPC对等连接只用于自定义VPC之间，产品上是否也是如此限制待确认。
- 3、kube-ovn中service是基于ovn lb实现，不依赖kube-proxy。
- 4、默认不支持跨VPC ClusterIP与NodePort svc访问，原因： kube-ovn 只在节点上创建到默认VPC下的子网的路由规则，所以用户自定义 VPC 下的子网从节点上是不通的。但是由于 k8s 的 SVC 功能依赖节点的转发，所以依赖 SVC 的功能无法工作。
- 5、网络架构图：



对比 社区 kube-ovn 区别:

- 问题: 不是所有节点都能访问外部网络
方案: 子网只支持 centralized gateway
- 问题: EAS-93408, 社区使用的 ovn 有私有 patch (修改 route priority), 我们的产品使用的是标准 ovn, 导致: 跨子网访问不通; eip pod 无法访问内部网络
方案: 去掉所有 AddStaticRoute()
- 问题: 因为 2 中 去掉了 src-ip 路由到 ovm0, 非 eip pod 无法访问外部网络
方案: 现在加入默认路由到 172.35.0.1, 以及根据 subnet natoutgoing 情况 设置 snat
- 问题: 因为 2 中 去掉了 src-ip 路由到 ovm0, pod 无法访问 coreDNS
方案: 加入 dst-ip 路由到 flannel 及 svc 地址到 ovm0

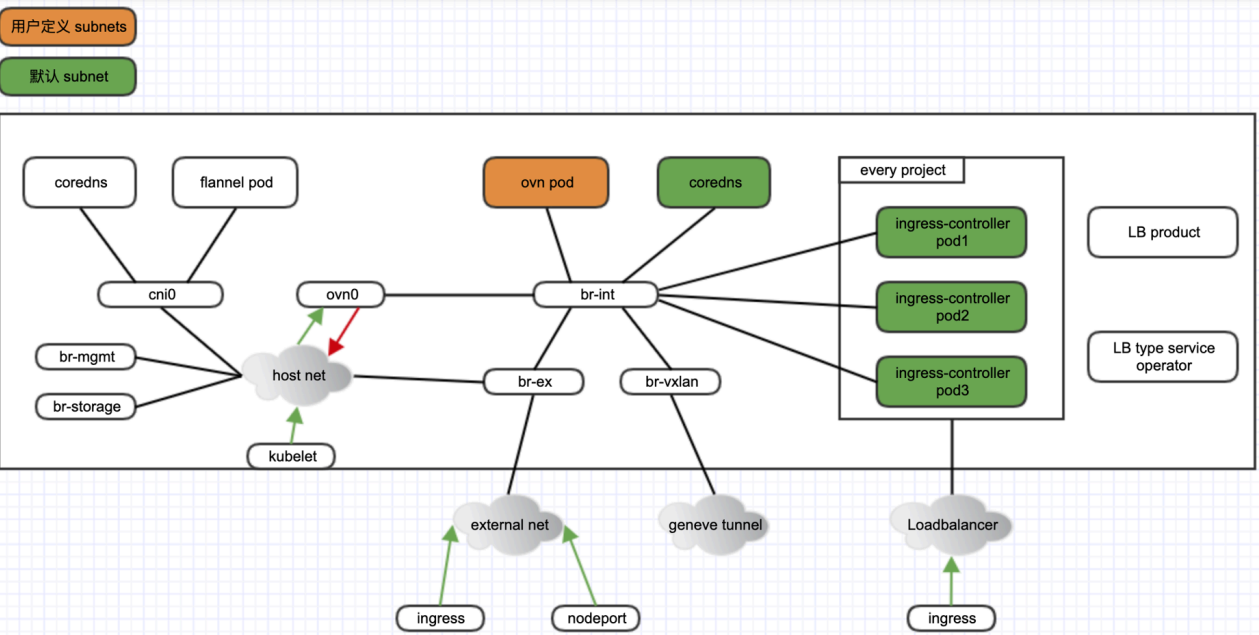
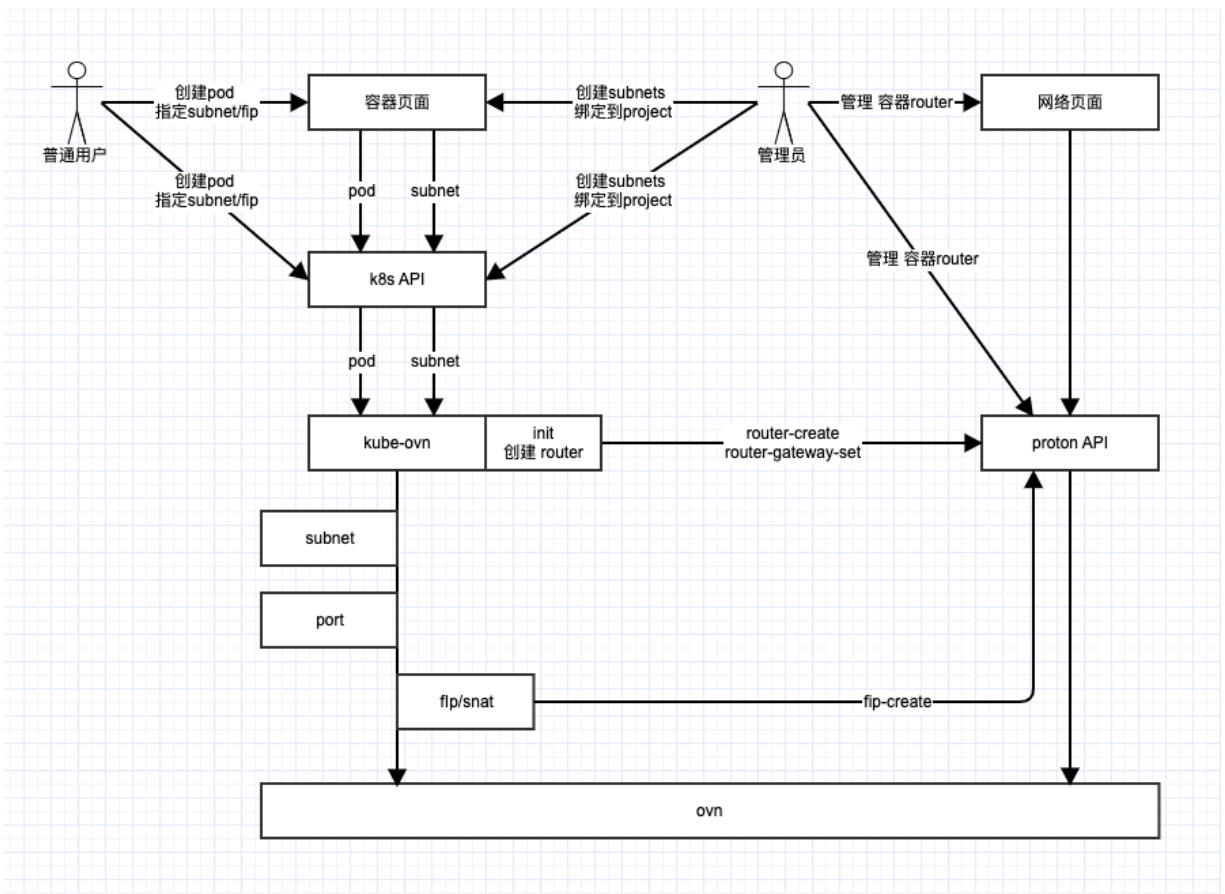
EIP:
client → 172.35.0.1 → br-ex(master) → br-int(master) → dnst → br-int(node) → pod
pod → br-int(node) → br-int(master) → undat → br-ex(master) → 172.35.0.1 → client

Pod 访问外网:
pod → br-int(node) → br-int(master) → snat → br-ex(master) → external network

Ingress:
client → master → ingress-controller → ovm0(master) → br-int(master) → br-int(node) → pod
pod → br-int(node) → br-int(master) → ovm0(master) → ingress-controller → client

nodePort:
client → master → ovm0(master) → br-int(master) → br-int(node) → pod
pod → br-int(node) → br-int(master) → ovm0(master) → ingress-controller → client

kube-ovn Pod 访问 flannel Pod:
pod → br-int(node) → ovm0(master) → host route → br-mgmt → cnl0 → pod



- 基本隔离策略/方法
放行: nodeport/ingress/探针 --> host --> ovn0 --> pod
禁止: pod --> ovn0 --> host/flannel 网络
方法: 不设置 访问ovn0的访问host/flannel dst的路由规则
- 允许 ovn pods 需要访问 k8s API, 复用kube-ovn配置的访问节点的策略路由
- 为ovn pods部署独立于控制平面的 coredns
- 为ovn pods 每个租户下 部署ingress-controller, 配置为LB类型的svc 对外暴露

6、网络方案：

kube-ovn 初始化阶段，调用 proton API 创建容器 router，作为容器VPC.

问题：router需要创建到 admin租户下

kube-ovn 根据环境网络规划，通过crd 创建subnets，只有管理员可以创建/管理subnets，subnets不被 proton 管理.

问题： a. subnet spec有 ns字段没有project 字段，所以为了页面过滤选择subnet 1. 可以加 project label，2. spec 加个 project字段. 建议labels 简单 够用 b. 管理员是否需要提供一个 跨租户share的 subnet，即不需要为每个租户至少创建一个网络. 建议提供，方便且符合容器使用习惯

kube-ovn根据用户选择的子网 独立创建port，port不被proton管理. 无需修改

容器使用FIP/ 指定IP做SNAT实现pod除外网管理，kube-ovn 使用fip 调用proton API去创建/占用这个 FIP，fip 的租户信息是当前用户租户，proton通过fip的router_id和tags属性判断fip被容器使用.

问题： 产品易用性不好，fip是全局资源且只在proton数据里，用户使用时需要先去proton fip页面看下哪个fip没有被使用，然后再通过容器页面创建消费. 建议方案：kube-ovn加一个FIP CRD，加个 goroutine 2s 去 proton同步一次，把已经使用fip同步到CR里，然后容器这里通过cr可以拿到已使用的 FIP列表，实现用户使用一套k8s API即可为容器使用FIP